

STUDENT RECORD MANAGEMENT SYSTEM

Assignment 1 – Mini Project: Console Record-Management Application

Course: MCA Semester I – Python Programming & Relational Database

Field	Details
Student Name	Vinit Surve
Roll Number	150097926014
Class / Division	MCA
Academic Year	2026–28
GitHub Repository	https://github.com/VinitSurve/student-record-management-system

Submitted as part of the academic requirements for the Python Programming & Relational Database course.

Table of Contents

1. Introduction
2. Problem Statement
3. Objectives
4. Project Description
5. Features
6. Technologies and Requirements
7. Python Concepts Demonstrated
8. System Design and Program Flow
9. Functional Modules
10. Data Storage and JSON Structure
11. Automatic Student ID Generation
12. Testing and Verification
13. Screenshots
14. Advantages and Limitations
15. Future Enhancements
16. Conclusion
17. GitHub Repository and Submission Contents

1. Introduction

Student Record Management System is a Python-based console application developed as a mini project for the MCA Semester I course Python Programming & Relational Database. The application provides a simple menu-driven interface for maintaining student information without requiring a database server or external Python packages.

The system stores student records using Python data structures while using a JSON file for persistent storage. Users can add new students, view existing records, search for a student by ID, update individual fields, delete records with confirmation, and exit the application safely.

The project is designed as a console application so that core Python programming concepts can be demonstrated clearly, including variables, data types, lists, dictionaries, conditions, loops, functions, exception handling, and file input/output.

The repository is publicly available on GitHub and contains the Python source code, persistent data file, README documentation, and application screenshots.

2. Problem Statement

Managing student information manually can become difficult when records need to be added, searched, updated, or deleted repeatedly. A small record-management application can simplify these operations by providing a consistent menu and storing the records in a structured format.

The problem addressed by this project is to design and implement a simple console-based system that can perform basic CRUD operations on student records while demonstrating fundamental Python programming techniques and persistent file storage.

3. Objectives

- Develop a menu-driven console application using Python.
- Implement operations for adding, viewing, searching, updating, and deleting student records.
- Use appropriate Python data types and data structures to represent student information.
- Organise the program using reusable functions.
- Use conditional statements and loops to control application flow.
- Persist records using a JSON file so data remains available after restarting the program.
- Demonstrate basic exception handling for file-related situations.
- Maintain a readable and executable project structure suitable for an academic mini project.

4. Project Description

The application maintains student records containing five fields: Student ID, Name, Age, Course, and Email. Records are kept in a Python list, with each student represented by a dictionary. The list is loaded from students.json when the application starts and written back to the file whenever a record is added, updated, or deleted.

The main menu provides six operations. The user selects an option by entering a number, after which the corresponding function performs the requested task and returns control to the main menu.

5. Features

Feature	Description
Add Student	Creates a new student record and saves it to JSON.
View All Students	Displays all stored student records.
Search Student	Searches for a student using the Student ID.
Update Student	Updates name, age, course, or email individually.
Delete Student	Deletes a selected record after user confirmation.
Persistent Storage	Stores records in students.json for reuse after restart.
Automatic IDs	Generates sequential IDs after the first starting ID is supplied.
Non-reuse of deleted IDs	New IDs continue from the highest existing ID plus one.
Menu-driven Interface	Provides a simple numbered console menu.

Application Menu

```
=====
      STUDENT RECORD MANAGEMENT
=====
1. Add Student
2. View All Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit
=====
```

6. Technologies and Requirements

Technology / Requirement	Purpose
Python 3.x	Programming language used to implement the application.
JSON	Structured, persistent storage for student records.
Python Standard Library	File handling and JSON processing through built-in modules.
Git / GitHub	Version control and source-code submission.
Console / Terminal	Runtime interface for the menu-driven application.

External packages: The application does not require external Python packages. It can be run directly with Python 3.x.

7. Python Concepts Demonstrated

Concept	Application in Project
Variables	Store student information, menu choices, and application state.
Data Types	Integers are used for IDs and ages; strings for names, courses, and emails.
Lists	The collection of student dictionaries is maintained in a list.
Dictionaries	Each student record is represented as a dictionary of fields.
Conditional Statements	Used for menu choices, record matching, confirmation, and branching.
Loops	The main menu repeats until Exit; loops also iterate through records.
Functions	Operations are separated into functions such as add_student() and delete_student().
Exception Handling	FileNotFoundError is handled when the data file does not exist.
File I/O	The JSON file is read at startup and written after changes.
JSON	Provides a structured text representation of the stored records.

8. System Design and Program Flow

The system follows a simple procedural design. A global list named students holds the records while the functions operate on that list. The application first loads existing records, then enters the main menu loop.

High-level flow

1. Start the Python program.
2. Call load_records() to load students.json if it exists.
3. Display the main menu.
4. Read the user's menu choice.
5. Call the function corresponding to the selected operation.
6. Save the modified list after Add, Update, or Delete.
7. Return to the menu until the user selects Exit.
8. Terminate the program.

Program Structure

```
main.py
|
|— load_records()
|— save_records()
|— display_menu()
|— add_student()
|— view_students()
|— search_student()
```

```
|— update_student()
|— delete_student()
|— main()
```

9. Functional Modules

load_records()

Loads existing student records from students.json. If the file does not exist, the application starts with an empty list.

save_records()

Writes the current student list to students.json using JSON formatting.

display_menu()

Displays the six available operations and provides the main user interface.

add_student()

Creates a student record. For the first record, the user supplies a starting ID. Subsequent records receive an automatically generated ID.

view_students()

Displays all records stored in memory. If no records exist, an appropriate message is shown.

search_student()

Accepts a Student ID, searches the list, and displays the matching record if found.

update_student()

Finds a student by ID and provides a submenu for updating the name, age, course, or email.

delete_student()

Finds a student by ID, displays the record, asks for confirmation, and removes the record when confirmed.

main()

Controls the repeating menu loop and calls the appropriate operation based on the user's selection.

10. Data Storage and JSON Structure

The project uses students.json as its persistent data file. This avoids the need for a separate database server while still demonstrating file-based persistence and structured data storage.

A student record is represented using a dictionary. Multiple student dictionaries are stored inside a list.

```
[
  {
    "id": 101,
    "name": "John Doe",
    "age": 22,
    "course": "MCA",
    "email": "john@example.com"
  }
]
```

Data persistence process

- At startup, load_records() reads students.json.
- The JSON content is converted into Python objects.
- Application operations modify the in-memory list.
- save_records() writes the updated list back to students.json.
- The saved records are available the next time the program starts.

11. Automatic Student ID Generation

The application includes an additional feature for automatic Student ID generation. When no student records exist, the user is asked to enter a starting Student ID.

```
Starting ID: 101

First student   → 101
Second student → 102
Third student  → 103
Fourth student → 104
```

For later additions, the next ID is calculated from the highest existing ID plus one. This allows the user to start numbering from a value appropriate to the assignment or dataset.

Deleted IDs are not reused. For example, if records 101, 102, and 103 exist and record 103 is deleted, the next newly added record receives 104 rather than reusing 103.

Update and Delete Safeguards

- Update operation uses a dedicated submenu so individual fields can be changed without rewriting the entire record.
- Delete operation displays the selected record before removal.
- Deletion requires an explicit y/n confirmation from the user.

12. Testing and Verification

The application was tested through functional scenarios covering the major assignment requirements and additional project features. A separate deterministic tester project was also used during development to exercise the GitHub source code without requiring an AI API.

Test Area	Expected Behaviour	Status / Evidence
Application launch	Program starts and shows the main menu.	Verified during functional testing
Add record	New student is added and saved.	Verified
View records	Stored records are displayed.	Verified
Search existing	Matching student is displayed.	Verified
Search missing	Student not found message is displayed.	Verified
Update record	Selected field is updated and saved.	Verified
Delete confirmation	Record is removed only after confirmation.	Verified

Delete cancellation	Record remains when deletion is cancelled.	Verified
Automatic IDs	IDs increment from the highest existing ID.	Verified
Persistence	Changes remain after restarting the application.	Verified
Missing JSON file	Application starts with an empty record list.	FileNotFoundException handled

Exception Handling Status

The current implementation explicitly handles `FileNotFoundException` when `students.json` is unavailable. During development testing, invalid numeric input such as entering letters where an ID or age is expected, and corrupted JSON content, were identified as cases that require additional `ValueError` and JSON parsing handling.

For an academic submission, these cases should be handled before final submission if the instructor expects exception handling for invalid input and file errors beyond a missing file.

13. Screenshots

The GitHub repository contains an application screenshot set in the `screenshots/` directory. The screenshots document the major user-facing operations of the console application.

File	Demonstrates
1. add-student.png	Adding a new student record.
2. view-all-students.png	Viewing all stored student records.
3. search-student.png	Searching for a student by ID.
4. update-student.png	Updating student information.
5. delete-student.png	Deleting a student record with confirmation.
6. exit-system.png	Exiting the application.

14. Advantages and Limitations

Advantages

- Simple console-based interface that is easy to understand and operate.
- Clear separation of operations using functions.
- Persistent storage without requiring a database server.
- Automatic Student ID generation reduces manual ID entry for later records.
- Individual-field update functionality avoids unnecessary data re-entry.
- Delete confirmation reduces accidental removal of records.
- Small project structure makes the code easy to study and demonstrate.

Limitations

- The application is console-based and does not provide a graphical interface.
- JSON file storage is suitable for a small assignment but is not intended for large-scale multi-user systems.
- The current implementation has limited input validation.
- Email format and age-range validation are not implemented.
- The application does not provide user authentication or role-based access.

15. Future Enhancements

- Add robust ValueError handling for all numeric inputs.
- Handle malformed JSON using JSONDecodeError with a safe recovery strategy.
- Validate email format and acceptable age ranges.
- Introduce a graphical or web-based user interface.
- Replace JSON storage with a relational database such as SQLite or PostgreSQL.
- Add authentication and role-based permissions.
- Add import/export functionality for CSV or other formats.
- Add automated unit tests and continuous integration.

16. Conclusion

The Student Record Management System demonstrates the development of a small menu-driven Python application for managing student records. It implements the core CRUD operations—add, view, search, update, and delete—while using functions, lists, dictionaries, conditional statements, loops, file handling, and JSON storage.

The project also includes practical extensions such as automatic Student ID generation, individual-field updates, delete confirmation, and persistent storage. The resulting application provides a compact demonstration of the programming concepts covered in the Python Programming & Relational Database course.

The GitHub repository serves as the source-code submission and contains the application files, README documentation, data file, and screenshots.

17. GitHub Repository and Submission Contents

Repository: <https://github.com/VinitSurve/student-record-management-system>

Repository Item	Purpose
main.py	Complete Python source code for the application.
students.json	Persistent student record data file.
README.md	Project description, features, concepts, usage instructions, and sample output.
screenshots/	Application screenshots demonstrating major operations.
Documentation PDF	Formal assignment report describing the project and implementation.

Final Submission Checklist

- Python source code is present in the GitHub repository.
- Required JSON data file is present.
- README.md is present and documents the project.
- Application screenshots are present in the screenshots/ directory.
- Formal assignment report is provided as a PDF.
- The application can be cloned and run using Python 3.x.

End of Report